

Lab1: Data Lab Report

24300810005 兰珂涵

1. 测试结果

执行 ./check_ops.py bits.c , 终端输出如下 :

```
Total points: 110/110
(base) ~/DataLab <main*> $ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 5/6 operators
copyByteWithin: 11/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 12/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 19/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 30/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 49/52 operators
floatScaleThreeHalves: 32/60 operators
floatRoundEven: 22/65 operators
float_i2f: 27/40 operators
bitCount: 36/40 operators
bitReverse: 17/34 operators
All 19 functions passed operator checks.
(base) ~/DataLab <main*> $
```

执行 ./btest , 19 个函数的 Errors 都为 0 , 总分为 110/110。

```
(base) ~/DataLab <main*> $ ./btest
Score  Rating  Errors  Function
1       1        0    signMask
2       2        0    bitXor
3       3        0    negativePart
4       4        0    copyByteWithin
4       4        0    logicalShift
4       4        0    swapNibblePairs
4       4        0    secondLowestZeroBit
5       5        0    oddParity
5       5        0    rotateRightBits
5       5        0    roundEvenPow2
5       5        0    midpointTowardFirst
7       7        0    isBetweenEitherOrder
7       7        0    mul5Sat
7       7        0    classifyAdd3
7       7        0    floatScaleThreeHalves
10      10        0    floatRoundEven
10      10        0    float_i2f
10      10        0    bitCount
10      10        0    bitReverse
Total points: 110/110
(base) ~/DataLab <main*> $
```

2. 每个函数的实现思路

以下截图对应 bits.c 中的完整函数。整数运算按实验规定的 32 位补码和算术右移规则解释。

P1 · signMask

bits.c | P1 signMask

```
148 int signMask(void) {
149     return 1 << 31;
150 }
```

思路：将整数 1 左移 31 位，把唯一的 1 移到最高位，其余位均为 0，得到 0x80000000。该操作按实验的 32 位位向量规则解释。

P2 · bitXor

bits.c | P2 bitXor

```
160 int bitXor(int x, int y) {
161     // x xor y = ~x y + ~y x = ~ ( ~(~x y) ~(~y x) )
162     return ~(~(~x & y) & ~(x & ~y));
163 }
```

思路：异或表示对应位不同，即 $(\sim x \& y)$ 或 $(x \& \sim y)$ 。由于本题只能使用 $\sim$ 和 $\&$ ，代码利用德摩根律把两部分的或运算改写为 $\sim(\sim A \& \sim B)$ ，从而实现异或。

P3 · negativePart

bits.c | P3 negativePart

```
173 int negativePart(int x){
174     return ~ (x + ~0) & (x >> 31);
175 }
```

思路： ~ 0 表示全 1，因此 $\sim(x + \sim 0)$ 等价于在 32 位补码中计算 $-x$ 。 $x \gg 31$ 在 x 为负时得到全 1，否则得到全 0；与运算据此选择返回 $-x$ 或 0。对 INT_MIN ，结果仍按 32 位补码保留为 0x80000000。

P4 · copyByteWithin

bits.c | P4 copyByteWithin

```
188 int copyByteWithin(int x, int src, int dst) {
189     int srcc = src << 3;
190     int dstt = dst << 3;
191     int srcMask = 255 << srcc;
192     int srcByte = x & srcMask;
193     int dstByte = ((srcByte >> srcc) & 0xFF) << dstt;
194     int dstMask = ~(255 << dstt);
195
196     return (x & dstMask) + dstByte;
197 }
```

思路：将源、目标字节编号左移 3 位，得到位偏移 srcc 和 dstt。先用 srcMask 提取源字节，右移后再与 0xFF，清除算术右移可能带入的高位；随后移到目标位置。dstMask 清空原目标字节，最后相加合并，两部分的有效位互不重叠。

P5 · logicalShift

bits.c | P5 logicalShift

```
208 int logicalShift(int x, int n) {
209     int aShift = x >> n ;
210     int y = 1 << 31;
211     int mask = ~((y >> n) << 1);
212     return aShift & mask;
213 }
```

思路：先执行算术右移，得到 aShift。从最高位为 1 的 y 出发，通过 $\sim((y \gg n) \ll 1)$ 构造低 $32-n$ 位为 1 的掩码，清除算术右移补入的符号位。n 为 0 时掩码为全 1，结果保持不变。

P6 · swapNibblePairs

bits.c | P6 swapNibblePairs

```
223 int swapNibblePairs(int x) {
224     int left = x << 4;
225     int right = x >> 4;
226     int mask = 240 + (240 << 8) + (240 << 16) + (240 << 24);
227     return (left & mask) + (right & (~mask));
228 }
```

思路：分别将 x 左移和右移 4 位。构造 $0xF0F0F0F0$ 作为 $mask$ ，从左移结果中保留各字节的高半字节，从右移结果中用 $\sim mask$ 保留各字节的低半字节，最后相加合并，实现每个字节内部高低 4 位交换。

P7 · secondLowestZeroBit

bits.c | P7 secondLowestZeroBit

```
239 int secondLowestZeroBit(int x) {
240     int y = ((x + 1) ^ x) | x;
241     int z = (y + 1) ^ y;
242     return z & (y + 1);
243 }
```

思路： $x+1$ 会翻转末尾连续的 1 以及其前方第一个 0。 $((x+1)^x)|x$ 因而把最低的 0 置为 1，得到 y 。随后 $(y+1)^y$ 标记下一次进位经过的位，再与 $y+1$ 相与，只保留 y 的最低 0 所在位置，也就是原 x 的第二低 0。若不足两个 0，结果为 0。

P8 · oddParity

bits.c | P8 oddParity

```
254 int oddParity(int x) {
255     int x1 = x ^ (x >> 16);
256     int x2 = x1 ^ (x1 >> 8);
257     int x3 = x2 ^ (x2 >> 4);
258     int x4 = x3 ^ (x3 >> 2);
259     int x5 = x4 ^ (x4 >> 1);
260
261     return ~x5 & 1;
262 }
```

思路：依次将数据与其右移 16、8、4、2、1 位的结果异或，把 32 位中 1 的数量的奇偶性折叠到最低位。最低位为 1 表示原数中有奇数个 1；题目要求偶数个 1 时返回 1，所以用 $\sim x5 \& 1$ 对这一位取反。

P9 · rotateRightBits

bits.c | P9 rotateRightBits

```
273 int rotateRightBits(int x, int n) {
274     int k = n & 31;
275     int ck = (~k + 1) & 31;
276     int right = x >> k;
277     int left = x << ck;
278     int mask = ~0 << ck; // n 1, 32 - n 0
279     return (left & mask) + (right & ~mask);
280 }
```

思路： $k=n\&31$ 将位移量按 32 取模， $ck=(\sim k+1)\&31$ 得到补位的左移量，避免移位 32 位。分别计算右移和左移结果，用 $mask=\sim 0\<\<ck$ 保留左移部分的高位、右移部分的低位，再相加。 k 为 0 时由左侧保留整个 x ，仍返回原值。

P10 · roundEvenPow2

bits.c | P10 roundEvenPow2

```
293 int roundEvenPow2(int x, int n) {
294     // if n bit is 100001, x >> n + 1
295     // if n bit is 0, x >> n
296     // if n bit is 100000, x >> n + 1 if x >> n & 1 else x >> n
297
298     int nbitis1 = (x << 1 >> n) & 1;
299     int nbitislandother0 = nbitis1 & !(x & ~(~0 << n >> 1));
300     int npluslbitis1 = (x >> n) & 1;
301
302     return ((x >> n) + ((nbitislandother0 & npluslbitis1) | (nbitis1 & !(nbitislandother0
303         < )))) << n;
303 }
```

思路：nbitis1 提取被舍弃部分的最高位，即半个舍入单位所在位；nbitislandother0 进一步判断低 n 位是否恰好只有该位为 1，也就是中点。npluslbitis1 表示保留商的奇偶性。大于中点时进位，恰在中点且商为奇数时进位，否则不进位。最后左移 n 位，将舍入后的商还原为 2^n 的倍数。

P11 · midpointTowardFirst

bits.c | P11 midpointTowardFirst

```
317 int midpointTowardFirst(int x, int y) {
318     int xSign = (x >> 31) & 1;
319     int ySign = (y >> 31) & 1;
320     int diff = y + (~x) + 1; // y - x
321     int diffSign = (diff >> 31) & 1;
322     // if positive overflow, diff >> 1, add 0, ySign is 0, x Sign is 1, diffSign is 1
323     // if negative overflow, diff >> 1, add 1, ySign is 1, x Sign is 0, diffSign is 0;
324     // if no overflow, diff >> 1, no need to reverse sign bit;
325
326     // To ensure toward first,
327     // if diff has half, if x > y, + 1
328     // which is y - x < 0, diffSign is 1 but not overflow or diffSign is 0 but overflow
329     int diffHalf = diff & 1;
330     int overflow = (xSign ^ ySign) & (diffSign ^ ySign);
331     int greater = diffSign ^ overflow;
332     int halfDiff = ((diff >> 1) ^ (overflow << 31)) + (greater & diffHalf);
333
334     return x + halfDiff;
335 }
```

思路：采用 $x+(y-x)/2$ 的形式。先计算差值 diff，利用 x、y 和差值的符号判断减法是否溢出。diffSign^overflow 恢复真实差值的正负，右移差值后通过翻转最高位修正溢出影响。若真实差值为负且为奇数，再加 1，使除以 2 的结果向 0 截断；最终加回 x，实现中点向第一个参数靠近。

P12 · isBetweenEitherOrder

bits.c | P12 isBetweenEitherOrder

```
347 int isBetweenEitherOrder(int x, int a, int b) {
348     // i.e. a >= x and x >= b, i.e. !(a < x) and !(x < b);
349     int xSign = (x >> 31) & 1;
350     int aSign = (a >> 31) & 1;
351     int adiff = a + (~x) + 1; // y - x
352     int adiffSign = (adiff >> 31) & 1;
353     int aoverflow = (xSign ^ aSign) & (adiffSign ^ aSign);
354     int agreater = adiffSign ^ aoverflow; // x > a
355
356     int bSign = (b >> 31) & 1;
357     int bdiff = x + (~b) + 1; // x - b
358     int bdiffSign = (bdiff >> 31) & 1;
359     int boverflow = (bSign ^ xSign) & (bdiffSign ^ xSign);
360     int bgreater = !(bdiffSign ^ boverflow); // x > b
361
362     int xisa = !(adiff);
363     int xisb = !(bdiff);
364
365     return xisa | xisb | (agreater ^ bgreater);
366 }
```

思路：分别计算 $a-x$ 和 $x-b$ ，通过操作数及结果的符号修正减法溢出，得到 $agreater$ (x 是否大于 a) 和 $bgreater$ (x 是否大于等于 b)。 x 不等于端点时，两者异或为 1 恰好表示 x 位于两个端点之间。再用 $xisa$ 、 $xisb$ 单独纳入端点，完成与端点顺序无关的闭区间判断。

P13 · mul5Sat

bits.c | P13 mul5Sat

```
377 int mul5Sat(int x) {
378     int y = x << 2;
379     int signx = (x >> 31) & 1;
380     int notoverflow0 = !(((y >> 2) + ~x) + 1);
381     int z = y + x;
382     int signz = (z >> 31) & 1;
383     int notoverflow1 = !(signz ^ signx);
384     int notoverflow = notoverflow0 & notoverflow1;
385     int tmp = (notoverflow << 31 >> 31);
386
387     return (z & tmp) + (((1 << 31) ^ (!signx << 31 >> 31)) & ~tmp);
388 }
```

思路：先左移两位得到 y ，再计算 $z=y+x$ ，对应乘以 5。将 y 算术右移两位并与 x 比较，判断乘以 4 是否已经溢出；同时比较 z 与 x 的符号，判断后续加法是否溢出。 tmp 在无溢出时为全 1，用来选取 z ；否则根据 x 的符号构造 INT_MAX 或 INT_MIN 并返回。

P14 · classifyAdd3

bits.c | P14 classifyAdd3

```
399 int classifyAdd3(int x, int y, int z) {
400     int tmin = 1 << 31;
401     int tmax = ~tmin;
402
403     int xsign = x >> 31;
404     int ysign = y >> 31;
405     int sumxy = x + y;
406     int sumsign = sumxy >> 31;
407
408     int overflow = ~(xsign ^ ysign) & (xsign ^ sumsign);
409     int sumneg = sumsign ^ overflow;
410     int sumpos = ~sumneg;
411
412     int negz = ~z + 1;
413     int sumlow = sumxy & tmax;
414
415     // greater: sumxy > INT_MAX - z
416     int maxz = tmax + negz;
417     int maxzsign = maxz >> 31;
418     int maxdiff = (maxz & tmax) + (~sumlow) + 1;
419     int lowgreater = maxdiff >> 31;
420
421     // if sum is positive and (sumsign is 1 and maxzsign is 0 or sumsign == maxzsign and
422     // lowbits sum greater)
423     int greater = sumpos & ((sumsign & !maxzsign) | (~(sumsign ^ maxzsign) & lowgreater));
424
425     // less x + y < INT_MIN - z
426     int minz = tmin + negz;
427     int minzsign = minz >> 31;
428     int mindiff = sumlow + ~(minz & tmax) + 1;
429     int lowless = mindiff >> 31;
430
431     int minzero = (!minz) << 31 >> 31;
432
433     // if sum is neg and (min is zero or (sumsign is 0 and minzsign is 1) or sumsign b
434     // ut lowbits sum less)
435     int less = sumneg & (minzero | (~sumsign & minzsign) | (~(sumsign ^ minzsign) & lowl
436     // ess));
437
438     return (greater & 1) + less;
439 }
```

思路：先计算 `sumxy`，用 `x`、`y` 和结果的符号检测加法溢出，从而恢复真实两数和的正负。然后将正溢出条件改写为 `x+y > INT_MAX-z`，负溢出条件改写为 `x+y < INT_MIN-z`。代码把两侧拆成低 31 位及最高位，低位差不会超出有符号整数范围，再结合真实符号和最高位关系完成比较；`minzero` 单独处理 `INT_MIN-z` 为 0 的边界。最后用 `(greater&1)+less` 返回 1、0 或 -1。

P15 · floatScaleThreeHalves

bits.c | P15 floatScaleThreeHalves

```
451 unsigned floatScaleThreeHalves(unsigned uf) {
452     unsigned sign = uf & 0x80000000;
453     unsigned exp = (uf >> 23) & 255;
454     unsigned sig = uf & 0x7FFFFFFF;
455     unsigned shift = 1;
456     unsigned rest;
457     unsigned half;
458
459     if (exp == 255) {
460         return uf;
461     }
462     if (exp) {
463         sig = sig | 0x800000;
464     } else {
465         exp = 1;
466     }
467
468     sig = sig + (sig << 1);
469     if (sig >= 0x2000000) {
470         shift = 2;
471         exp = exp + 1;
472     }
473     rest = sig & ((1 << shift) - 1);
474     half = 1 << (shift - 1);
475     sig = sig >> shift;
476     if (rest > half || (rest == half && (sig & 1))) {
477         sig = sig + 1;
478     }
479     if (sig >= 0x1000000) {
480         sig = sig >> 1;
481         exp = exp + 1;
482     }
483     if (exp >= 255) {
484         return sign | 0x7F800000;
485     }
486     if (sig < 0x800000) {
487         exp = 0;
488     }
489     return sign | (exp << 23) | (sig & 0x7FFFFFFF);
490 }
```

思路：提取符号、阶码和小数部分；NaN 与无穷大直接返回。规格化数补上隐含的最高位 1，非规格化数不补该位，但暂用阶码 1 对齐指数。将有效数精确乘以 3，再右移 1 位完成乘以 1.5；若有效数过大，则改为右移 2 位并增加阶码。根据被舍弃位执行最近偶数舍入，处理进位、溢出为无穷大及非规格化结果，最后拼回编码并保留符号。

P16 · floatRoundEven

bits.c | P16 floatRoundEven

```
504 unsigned floatRoundEven(unsigned uf) {
505     unsigned sign = uf & 0x80000000;
506     unsigned exp = (uf >> 23) & 255;
507     unsigned shift;
508     unsigned unit;
509     unsigned mask;
510     unsigned rest;
511     unsigned half;
512     unsigned rounded;
513
514
515     if (exp >= 150) {
516         return uf;
517     }
518     if (exp < 126) {
519         return sign;
520     }
521     if (exp == 126) {
522         if (uf & 0x7FFFFFFF) {
523             return sign | 0x3F800000;
524         }
525         return sign;
526     }
527     shift = 150 - exp;
528     unit = 1 << shift;
529     mask = unit - 1;
530     rest = uf & mask;
531     half = unit >> 1;
532     rounded = uf & ~mask;
533     if (rest > half || (rest == half && ((uf >> shift) & 1))) {
534         rounded = rounded + unit;
535     }
536     return rounded;
537 }
```

思路：阶码不小于 150 时，有限数已无小数位，NaN 和无穷大也在该分支原样返回。阶码小于 126

时舍入到带符号的零；等于 126 时，恰好 ± 0.5 舍入到零，其余舍入到 ± 1 。其他情况下，由 $150 - \text{exp}$ 确定小数位数，清除这些位得到截断值，再比较余数与半单位；超过中点或中点且保留整数为奇数时，给编码加一个整数单位。

P17 · float_i2f

bits.c | P17 float_i2f

```
549 unsigned float_i2f(int x) {
550     unsigned sign = x & 0x80000000;
551     unsigned mag = x;
552     unsigned scan;
553     unsigned exp = 0;
554     unsigned sig;
555     unsigned shift;
556     unsigned rest;
557     unsigned half;
558
559     if (!x) {
560         return 0;
561     }
562     if (sign) {
563         mag = ~mag + 1;
564     }
565     scan = mag;
566     while (scan >> 1) {
567         scan = scan >> 1;
568         exp = exp + 1;
569     }
570     if (exp <= 23) {
571         sig = mag << (23 - exp);
572     } else {
573         shift = exp - 23;
574         sig = mag >> shift;
575         rest = mag & ((1 << shift) - 1);
576         half = 1 << (shift - 1);
577         if (rest > half || (rest == half && (sig & 1))) {
578             sig = sig + 1;
579         }
580     }
581     return sign | (((exp + 126) << 23) + sig);
582 }
```

思路：先处理 0，再提取符号；负数通过无符号的取反加一求幅值，兼容 INT_MIN。循环右移 scan，找到最高 1 的位置 exp。幅值不超过 24 位时左移对齐，否则保留高 24 位，并按余数执行最近偶数舍入。最后将仍含隐含位的 sig 加到 (exp+126)<<23 上：隐含位补足阶码偏置，舍入产生的额外进位也会自然传入阶码。

P18 · bitCount

bits.c | P18 bitCount

```
594 int bitCount(int x) {
595     // 2 bits sum -> 4 bits sum ...
596     int mask16 = 255 + (255 << 8);
597     int mask8 = 255 + (255 << 16);
598     int half4 = 15 | (15 << 8);
599     int half2 = 51 | (51 << 8);
600     int half1 = 85 | (85 << 8);
601     int mask4 = half4 | (half4 << 16);
602     int mask2 = half2 | (half2 << 16);
603     int mask1 = half1 | (half1 << 16);
604
605     int x1 = (x & mask1) + ((x >> 1) & mask1);
606     int x2 = (x1 & mask2) + ((x1 >> 2) & mask2);
607     int x3 = (x2 & mask4) + ((x2 >> 4) & mask4);
608     int x4 = (x3 & mask8) + ((x3 >> 8) & mask8);
609
610     return (x4 & mask16) + ((x4 >> 16) & mask16);
611 }
```

思路：使用分组求和：依次统计每 2、4、8、16 位中的 1 的数量，最后合并两个 16 位组。每一步都通过掩码提取相邻两组，移位对齐后相加。half4、half2、half1 先构造 16 位模式，再左移 16 位并按位或，分别生成 0x0F0F0F0F、0x33333333 和 0x55555555，减少重复构造掩码的操作。

P19 · bitReverse

bits.c | P19 bitReverse

```
622 int bitReverse(int x)
623 {
624     // reverse 16,16, 8,8,8,8 ...
625     int mask16 = 255 + (255 << 8);
626     int mask8 = 255 + (255 << 16);
627     int mask4 = 15 + (15 << 8);
628     mask4 = mask4 + (mask4 << 16);
629     int mask2 = 51 + (51 << 8);
630     mask2 = mask2 + (mask2 << 16);
631     int mask1 = 85 + (85 << 8);
632     mask1 = mask1 + (mask1 << 16);
633
634     int x1 = ((x & mask16) << 16) + (x >> 16 & mask16);
635     int x2 = ((x1 & mask8) << 8) + (x1 >> 8 & mask8);
636     int x3 = ((x2 & mask4) << 4) + (x2 >> 4 & mask4);
637     int x4 = ((x3 & mask2) << 2) + (x3 >> 2 & mask2);
638
639     return ((x4 & mask1) << 1) + (x4 >> 1 & mask1);
640 }
```

思路：按代码顺序，先交换高低 16 位，再在每个 16 位组内交换两个字节，随后交换相邻的 4 位、2 位和 1 位组。每一步用对应掩码选出要左移和右移的部分，移到新位置后相加；两部分有效位互不重叠，右移结果上的掩码还能清除符号扩展。五级交换完成后，原来的第 i 位被移动到第 $31-i$ 位。